



TED UNIVERSITY

CMPE 492 – Senior Project II Test Plan Report

Team Members

Tolga AKYOL

Yiğit AYDIN

Yusuf KAYNAKÇI

Supervisor

Dr. Evren Coşkun

Jury Members

Prof. Dr. Tansel Dökeroğlu, Dr. Deniz Cantürk

Document Version: v1.0, Date: 10/04/2026

SmartLeaf

1. Introduction

This Test Plan defines the scope, approach, and schedule of the testing activities for the SmartLeaf system. It identifies the components and features to be tested, describes the testing methodology, assigns responsibilities, and outlines potential risks.

SmartLeaf is an AI-based plant health analysis system that enables users to evaluate plant conditions through leaf images. The system provides outputs such as disease classification, confidence scores, health scores, and possible nutrient deficiency indications through a web-based interface.

The system follows a modular architecture consisting of frontend, backend, machine learning, and reporting components. The testing activities focus on validating both individual components and their interactions within the overall system, including data flow, error handling, and end-to-end behavior.

The objective of testing is to verify that SmartLeaf satisfies its functional and non-functional requirements. Functional verification includes core features such as image upload, preprocessing, classification, result presentation, and report generation, while non-functional verification covers performance, usability, reliability, portability, and privacy.

Since SmartLeaf is a decision-support system, testing also ensures that outputs are presented responsibly. The system must communicate uncertainty clearly, handle invalid inputs safely, and avoid presenting results as definitive diagnoses.

This document outlines the testing strategy, including unit, integration, system, performance, and usability testing. Test results will be documented and included in the final report.

2. Test Objectives

The primary objective of testing is to verify that the SmartLeaf system operates correctly according to its functional and non-functional requirements, and that it delivers reliable outputs to end users.

From a functional perspective, testing ensures that core features—such as image upload and validation, preprocessing, disease classification, confidence and health score generation, nutrient deficiency indication, result presentation, and PDF report generation—are correctly implemented. The system must also handle invalid inputs, unclear images, and low-confidence predictions appropriately.

From a non-functional perspective, testing evaluates system qualities including performance, usability, reliability, portability, and security. The system should respond within acceptable time limits, provide a user-friendly interface, behave consistently across executions, and function correctly across different devices and browsers.

Testing also verifies correct interaction between system components. This includes ensuring proper data flow and communication between frontend, backend, machine learning, and reporting modules, as well as correct end-to-end behavior.

In addition, testing evaluates how the system behaves under error conditions such as invalid inputs, corrupted data, or subsystem failures. The system must handle such situations gracefully and provide clear feedback without crashing.

Since SmartLeaf is a decision-support system, testing also ensures that outputs are presented responsibly. The system must clearly communicate uncertainty and avoid presenting results as definitive diagnoses.

Finally, testing establishes a structured basis for evaluation by executing defined test cases and recording pass/fail results for inclusion in the final report.

3. Test Scope

This section defines the boundaries of testing activities for the SmartLeaf system, specifying which components and features are included and which are excluded.

3.1 In-Scope Features

Testing covers the core functionalities and system components of SmartLeaf, including the frontend, backend API, machine learning subsystem, and reporting module, as well as their integration.

The following features are included in the test scope:

- **Image Upload and Validation**

Verify that supported image formats are accepted and invalid or oversized files are rejected.

- **Image Preprocessing**

Ensure that images are correctly prepared for model inference (e.g., resizing and normalization).

- **Disease Classification and Confidence Output**

Verify that the system returns a valid prediction and corresponding confidence score.

- **Health Score Generation**

Ensure that a health score is calculated within the expected range.

- **Nutrient Deficiency Indication**

Verify that the system provides non-definitive suggestions based on visual features.

- **Result Presentation (Frontend)**

Ensure that analysis results are displayed clearly and correctly in the user interface.

- PDF Report Generation and Export

Verify that users can generate and download a structured report.

- Error Handling and User Feedback

Ensure that the system provides clear feedback for invalid inputs and low-confidence results.

- Data Handling and Privacy Behavior

Verify that uploaded images are processed temporarily and not stored unnecessarily.

- Health Check and System Status Endpoint

Ensure that system status can be retrieved from the backend.

- End-to-End System Workflow

Validate the complete process from image upload to result display and report generation.

- Subsystem Integration

Verify correct communication and data flow between system components.

These features represent the core functionality of SmartLeaf and are essential for validating system behavior.

3.2 Out-of-Scope Features

The following aspects are not included in the current testing scope:

- Large-scale production testing (e.g., high concurrency or stress testing)
- Extensive real-world agricultural validation across diverse conditions
- Future system extensions (e.g., growth tracking, multi-crop support)
- Long-term data storage and analytics features
- Advanced security mechanisms such as full authentication and authorization

These limitations reflect the scope of the current project phase.

4. Test Items

This section identifies the main components of the SmartLeaf system that will be tested. The system follows a modular architecture consisting of frontend, backend, machine learning, and supporting components.

The following items are included in the testing process:

- Frontend Application (User Interface)

Handles user interaction, including image upload and result display.

- Backend API

Manages request handling, input validation, and response generation.

- Machine Learning Subsystem

Performs image processing and prediction.

- Image Preprocessing Module

Prepares input images for model inference.

- Analysis and Business Logic Services

Coordinates the overall analysis workflow and system operations.

- Reporting and PDF Generation Module

Generates downloadable reports based on analysis results.

- Data Handling Logic

Manages temporary processing and deletion of uploaded data.

- System Integration (End-to-End Workflow)

Represents the complete interaction between all system components.

These components will be tested at different levels, including unit, integration, and system testing.

5. Features to Be Tested

This section describes the functionalities and system behaviors to be verified during testing.

5.1 Functional Features

The following core functionalities of SmartLeaf will be tested:

- Image Upload and Validation

Accept supported formats and reject invalid or corrupted files.

- Image Preprocessing

Prepare input images for model inference.

- Disease Classification

Generate a prediction for the uploaded image.

- Confidence Score Generation

Provide a confidence value for each prediction.

- Health Score Calculation

Produce a health score within the expected range.

- Nutrient Deficiency Indication

Provide non-definitive suggestions based on visual features.

- Result Presentation

Display analysis results clearly in the user interface.

- PDF Report Generation

Generate and download a report containing the results.

- Error Handling and Feedback

Provide clear feedback for invalid inputs and uncertain results.

- Health Check Endpoint

Provide system status through the backend.

- End-to-End Workflow Execution

Ensure the complete process runs without failure.

5.2 Non-Functional Features

The following system qualities will be evaluated:

- Performance

Complete analysis within an acceptable time.

- Accuracy and Consistency

Produce stable and consistent results.

- Usability

Provide a simple and intuitive user experience.

- Reliability

Operate without crashes and handle errors properly.

- Scalability (Basic Level)

Support multiple users under moderate load.

- Security and Privacy

Protect user data and avoid unnecessary storage.

- Portability

Work across different devices and browsers.

- Maintainability

Support future updates through a modular structure.

6. Test Strategy/Methodology

This section outlines the overall testing approach for the SmartLeaf system. The strategy covers multiple levels of testing, from individual components to end-to-end system behavior.

The testing process includes unit testing, integration testing, system testing, performance testing, and user-oriented evaluation to ensure that the system functions correctly and meets expected performance and usability standards.

6.1 Unit Testing

Unit testing focuses on testing individual components and functions in isolation. Each module is tested independently before integration with other components.

The following components will be subject to unit testing:

- Backend service methods responsible for input validation and analysis workflow
- Image preprocessing functions (e.g., resizing, normalization, segmentation)
- Machine learning service functions for prediction and confidence generation
- Reporting functions responsible for formatting results and generating PDF outputs
- Controller-level functions such as request handling and health-check endpoints

Unit tests cover input-output behavior, edge cases, and error handling at the component level.

6.2 Integration Testing

Integration testing focuses on verifying the interaction between different subsystems of the SmartLeaf architecture, including the frontend, backend, machine learning, and reporting components.

Integration testing will cover:

- Communication between frontend and backend API (request–response cycle)
- Backend interaction with the machine learning subsystem
- Data flow from preprocessing to prediction and result formatting
- Interaction between backend services and reporting/PDF generation module

The goal is to ensure that components exchange data correctly and function as a cohesive system.

6.3 System Testing

System testing focuses on the complete SmartLeaf system, evaluating end-to-end functionality from the user’s perspective.

System testing will include scenario-based testing such as:

- Uploading a valid image and receiving analysis results
- Handling invalid or non-leaf images with appropriate feedback
- Processing low-confidence cases and displaying warnings
- Generating and downloading PDF reports

The goal is to ensure that the overall workflow—from user input to final output—operates correctly and consistently.

6.4 Performance Testing

Performance testing focuses on the system’s responsiveness and efficiency under normal operating conditions.

The following aspects will be tested:

- Time required for image analysis (from upload to result display)
- Response time for backend API operations
- System behavior under moderate user load

The goal is to ensure that analysis is completed within acceptable time limits and that the system remains responsive during use.

6.5 Usability and User Acceptance Testing

Usability testing focuses on how easily users can interact with the system. Since SmartLeaf targets non-technical users, the interface should be simple and intuitive.

This testing will focus on:

- Ease of navigation and interaction flow
- Clarity of instructions and feedback messages
- Number of steps required to complete analysis
- Readability of results

User acceptance testing (UAT) will involve a small group of users interacting with the system and providing feedback on usability and functionality. This helps ensure that the system meets user expectations in real-world scenarios.

6.6 Error Handling and Negative Testing

This testing focuses on how the system behaves under invalid or unexpected conditions.

The following cases will be evaluated:

- Invalid or unsupported file formats
- Corrupted or low-quality images
- Failures in machine learning inference
- Network or communication issues

The system should handle these cases gracefully, provide clear error messages, and avoid crashes.

6.7 Security and Privacy Testing (Basic Level)

Basic security and privacy aspects of the system will be tested within the project scope.

This includes:

- Secure data transmission (HTTPS)
- Avoiding permanent storage of uploaded images
- Input validation to prevent invalid or harmful inputs

This level of testing ensures that the system meets basic privacy and data-handling expectations.

7. Test Environment

This section describes the hardware, software, tools, and conditions under which the SmartLeaf system will be tested. The goal of defining the test environment is to ensure that testing is conducted in a controlled and consistent setting that closely reflects the intended deployment conditions of the system.

7.1 System Architecture Environment

SmartLeaf follows a layered architecture consisting of frontend, backend, machine learning, and reporting components. The testing environment will reflect this structure to ensure accurate validation of subsystem interactions.

- Frontend Layer:

A web-based user interface implemented using modern JavaScript frameworks (e.g., React/Next.js), accessible through standard web browsers.

- Backend Layer:

A RESTful API service responsible for request handling, input validation, and orchestration of system operations.

- Machine Learning Layer:

A Python-based inference service responsible for preprocessing input images and generating predictions using a trained model.

- Reporting Module:

A component responsible for formatting results and generating PDF outputs for download.

These components will be tested both independently and as an integrated system.

7.2 Hardware Environment

Testing will be performed on standard user-level hardware to reflect realistic usage conditions:

- Personal computers (laptops/desktops)
- Mobile devices (smartphones/tablets)
- Standard internet connection

No specialized hardware (e.g., GPUs) is required for system usage, as inference is handled on the server side.

7.3 Software Environment

The following software environment will be used during testing:

- Operating Systems: Windows, macOS, and mobile operating systems (iOS/Android)
- Web Browsers: Google Chrome, Mozilla Firefox, Safari, and Microsoft Edge
- Backend Runtime: Server-side environment supporting RESTful API execution (e.g., Node.js or Python-based backend)
- Machine Learning Runtime: Python environment with required libraries for image processing and model inference
- PDF Generation Tools: Libraries or modules used for report creation and export

This ensures cross-platform compatibility and verifies that the system works correctly across different environments.

7.4 Test Data

A diverse set of test data will be used to evaluate system behavior under different conditions:

- Valid plant leaf images (healthy and diseased)
- Low-quality or blurred images
- Non-leaf or irrelevant images
- Unsupported file formats
- Oversized image files
- Repeated identical inputs for consistency testing

This variety ensures that both normal and edge-case scenarios are covered during testing.

7.5 Network Environment

Testing will be conducted under standard network conditions to simulate real-world usage:

- Stable internet connection for normal operation
- Occasional network interruptions to evaluate system behavior under network interruptions

The system must handle network-related issues gracefully and maintain correct behavior during communication between frontend, backend, and machine learning services.

8. Test Data

This section describes the types of data that will be used during testing to evaluate the functionality and performance of the SmartLeaf system. Test data is designed to cover both normal operating conditions and edge cases to ensure comprehensive system validation.

8.1 Valid Input Data

The following valid data will be used to verify normal system behavior:

- Healthy leaf images

Images representing healthy plant leaves to verify correct classification and high health score outputs.

- Diseased leaf images

Images containing visible symptoms of plant diseases (e.g., early blight, late blight) to verify classification accuracy and confidence score generation.

- Standard-quality images

Images with acceptable resolution and clarity to test normal preprocessing and inference behavior.

- Repeated identical inputs

The same image used multiple times to verify consistency and reproducibility of results.

These inputs are expected to produce valid outputs without triggering errors.

8.2 Invalid and Edge-Case Data

To evaluate system behavior and error handling, the following test data will be used:

- Unsupported file formats

Files such as .txt, .pdf, or other non-image formats to verify input validation.

- Oversized images

Files exceeding defined size limits to test system constraints and validation mechanisms.

- Corrupted or unreadable images

Damaged image files to verify error detection and handling.

- Non-leaf images

Images that do not contain plant leaves (e.g., objects, landscapes) to test system response to irrelevant inputs.

- Low-quality or blurred images

Images with poor resolution or noise to evaluate system behavior under uncertain conditions.

These inputs should trigger appropriate error messages or warnings without causing system failure.

8.3 Boundary and Performance Data

The following data will be used to evaluate system limits and performance:

- Images near size limits

Files close to the maximum allowed size to test boundary conditions.

- Images with varying resolutions

Low-resolution and high-resolution images to test preprocessing and performance impact.

- Multiple consecutive requests

Repeated submissions to evaluate system responsiveness and stability under continuous usage.

These tests ensure that the system operates correctly within defined constraints and maintains acceptable performance levels.

8.4 Data Source

Test data will be obtained from:

- Publicly available datasets (e.g., PlantVillage dataset)
- Sample images collected during development
- Manually generated test cases for edge scenarios

This combination ensures diversity in input data and supports realistic testing conditions for the SmartLeaf system.

9. Entry and Exit Criteria

This section defines the conditions that must be satisfied before testing begins (entry criteria) and the conditions that must be met for testing to be considered complete (exit criteria). These criteria ensure that testing is conducted in a controlled and systematic manner and that results are meaningful and reliable.

9.1 Entry Criteria

Testing activities will begin only when the following conditions are satisfied:

- **System Availability**

The core components of the SmartLeaf system (frontend, backend API, and machine learning subsystem) are implemented and accessible.

- **Basic Functionality Implemented**

Essential features such as image upload, analysis request handling, and result generation are operational.

- **Stable Build**

A stable version of the system is deployed or runnable in a testing environment without critical blocking issues.

- **Test Environment Prepared**

The required hardware, software, and network conditions are set up and verified.

- **Test Data Prepared**

Valid and invalid test datasets are available for executing different test scenarios.

- **Test Cases Defined**

Test cases with unique test IDs, steps, and expected results are documented and ready for execution, as required by the test plan guidelines.

- **Dependencies Available**

All required libraries, APIs, and external services (e.g., ML model, PDF generation module) are correctly configured.

Only after these conditions are met will formal testing activities begin.

9.2 Exit Criteria

Testing will be considered complete when the following conditions are satisfied:

- Test Case Execution Completed

All defined test cases have been executed and their results (pass/fail) have been recorded.

- Critical Defects Resolved

All critical and high-priority defects have been identified and resolved.

- Acceptable Defect Level

Remaining defects, if any, are minor and do not significantly impact system functionality or user experience.

- End-to-End Workflow Verified

The complete system workflow (image upload → analysis → result display → PDF export) has been successfully validated.

- System Stability Achieved

The system operates without crashes or major failures during testing.

- Performance Requirements Met (Basic Level)

The system demonstrates acceptable response times under normal conditions.

- Test Results Documented

All test outcomes are documented in a structured manner to be included in the final project report, including pass/fail status and observations.

- Supervisor/Team Approval

The testing process and results are reviewed and approved by the project team and, if required, by the supervisor.

10. Roles and Responsibilities

This section defines the roles and responsibilities of the team members involved in the testing process of the SmartLeaf system. Clear assignment of responsibilities ensures that testing activities are organized, traceable, and efficiently executed.

10.1 Team Roles

The SmartLeaf project is developed by a three-member team. Testing responsibilities are distributed based on system components and areas of development.

Tolga Akyol

Responsible for frontend and overall system integration testing. This includes:

- Testing user interface functionality (image upload, result display, navigation)
- Verifying frontend–backend communication
- Executing end-to-end system tests
- Documenting and organizing test results

Yiğit Aydın

Responsible for backend and machine learning subsystem testing. This includes:

- Testing backend API endpoints and request handling
- Verifying image preprocessing and machine learning inference
- Validating prediction outputs and confidence scores
- Performing performance-related testing

Yusuf Kaynakçı

Responsible for data handling, reporting, and system reliability testing. This includes:

- Testing PDF generation and report formatting
- Verifying data handling behavior (temporary storage and deletion of images)
- Testing error handling and robustness under invalid inputs
- Supporting integration testing and validating system stability

10.2 Shared Responsibilities

All team members collaboratively participate in the following activities:

- Designing and defining test cases with unique test IDs
- Preparing and organizing test data sets
- Executing integration and system-level tests

- Identifying, reporting, and resolving defects
- Reviewing testing outcomes and ensuring completeness
- Preparing documentation for inclusion in the final report

10.3 Supervisor Role

The project supervisor provides guidance and feedback throughout the testing process. Responsibilities include:

- Reviewing the test plan and testing methodology
- Providing feedback on test coverage and completeness
- Evaluating the quality and effectiveness of testing activities

10.4 External Users (Optional – Beta Testing)

A small group of external users may participate in usability and acceptance testing. Their role includes:

- Interacting with the system under real usage conditions
- Providing feedback on usability and clarity
- Reporting issues encountered during testing

11. Test Schedule

This section outlines the planned timeline for testing activities in the SmartLeaf project. The schedule is designed to align with the CMPE 492 course timeline and ensures that all testing phases are completed before the Test Plan submission deadline and subsequent final report preparation.

Testing activities are organized into phases to allow gradual verification of system components, starting from unit-level validation and progressing toward full system testing.

11.1 Testing Phases and Timeline

The testing process will be carried out according to the following schedule:

Phase 1: Test Planning and Preparation

- Define test strategy and methodology
- Prepare test cases with unique test IDs
- Collect and organize test data
- Set up testing environment

Phase 2: Unit Testing

- Test individual components such as preprocessing functions, backend services, and reporting modules
- Verify correctness of input-output behavior and edge cases

Phase 3: Integration Testing

- Test interactions between frontend, backend, machine learning, and reporting components
- Validate data flow and communication between subsystems

Phase 4: System Testing

- Execute end-to-end test scenarios (image upload → analysis → result display → PDF export)
- Verify complete system functionality from a user perspective

Phase 5: Performance and Usability Testing

- Evaluate system response time and stability
- Conduct usability testing with real users
- Collect feedback and identify improvements

Phase 6: Final Testing and Review

- Re-run critical test cases after bug fixes
- Ensure all test cases are executed and documented
- Prepare results for inclusion in the final report

11.2 Timeline Alignment

The testing activities are planned to be completed within the project timeline, with the Test Plan delivered by the required deadline and testing continuing in parallel with implementation improvements.

- Early phases (planning and unit testing) are conducted before and around the Test Plan submission
- Integration and system testing continue after core features are implemented
- Final testing results will be documented and included in the final report as required by the course guidelines

12. Risks and Mitigation

This section identifies potential risks that may affect the testing process and overall system quality of SmartLeaf, along with strategies to mitigate these risks. Identifying risks in advance helps ensure that testing activities remain effective and that potential issues are addressed proactively.

12.1 Technical Risks

Machine Learning Accuracy Limitations

The classification model may not achieve the desired accuracy due to limited or imbalanced training data.

Mitigation:

- Use validated datasets (e.g., PlantVillage)
- Evaluate model performance on different test samples
- Clearly communicate uncertainty through confidence scores

Model Inference Errors or Failures

The machine learning subsystem may fail during preprocessing or inference.

Mitigation:

- Implement error handling in the backend
- Validate input before processing
- Provide fallback error messages to users

Performance Issues (High Latency)

Image processing and inference may take longer than expected.

Mitigation:

- Limit input image size
- Use optimized or lightweight models
- Monitor response times during testing

12.2 Integration Risks

Subsystem Communication Failures

Errors may occur in communication between frontend, backend, and machine learning components.

Mitigation:

- Use well-defined API interfaces
- Perform integration testing early and continuously

- Implement health-check endpoints for monitoring system status

Data Flow Inconsistencies

Incorrect data formatting or transformation between components may lead to incorrect outputs.

Mitigation:

- Standardize input/output formats (e.g., JSON structures)
- Validate data at each stage of processing
- Use consistent interface definitions

12.3 Data and Input Risks

Invalid or Unexpected User Inputs

Users may upload unsupported, corrupted, or irrelevant images.

Mitigation:

- Implement strict input validation (file type, size, format)
- Provide clear error messages and guidance

Low-Quality or Ambiguous Images

Poor-quality images may lead to unreliable predictions.

Mitigation:

- Detect low-confidence results and warn users
- Encourage users to upload clearer images

12.4 Usability Risks

User Misinterpretation of Results

Users may treat system outputs as definitive diagnoses instead of suggestions.

Mitigation:

- Clearly display disclaimers and confidence scores
- Emphasize that results are decision-support only

Complex or Confusing Interface

A poorly designed interface may reduce usability.

Mitigation:

- Keep interaction flow simple

- Use clear labels and feedback messages
- Conduct usability testing with real users

12.5 Security and Privacy Risks

Improper Data Handling

Uploaded images may be unintentionally stored or exposed.

Mitigation:

- Process images temporarily and avoid permanent storage
- Use secure communication (HTTPS)
- Follow basic data protection practices

12.6 Project and Schedule Risks

Limited Development Time

As a student project, time constraints may limit testing coverage.

Mitigation:

- Prioritize critical test cases
- Focus on core system functionality
- Perform incremental testing throughout development

Incomplete Testing Coverage

Not all edge cases may be fully tested.

Mitigation:

- Use diverse test data
- Include both normal and edge-case scenarios
- Document limitations clearly in the final report

12.7 Summary

The identified risks highlight potential challenges in system performance, integration, usability, and data handling. By applying the mitigation strategies outlined above, the SmartLeaf testing process aims to reduce the impact of these risks and ensure a reliable and robust system.

13. Test Cases

This section defines the test cases that will be executed to verify the functionality and behavior of the SmartLeaf system. Each test case is labeled with a unique Test ID and includes a description, execution steps, and expected results.

TC-001 – Valid Image Upload

Type: Functional

Description: Verify that a valid plant leaf image can be uploaded successfully.

Steps:

1. Open the SmartLeaf web interface
2. Upload a valid JPEG/PNG image

Expected Result: The image is accepted and sent to the backend without errors.

TC-002 – Unsupported File Format

Type: Negative

Description: Verify that unsupported file formats are rejected.

Steps:

1. Upload a .txt or .pdf file

Expected Result: The system rejects the file and displays an error message.

TC-003 – Oversized File Upload

Type: Boundary

Description: Verify that file size limits are enforced.

Steps:

1. Upload an image larger than allowed size

Expected Result: The system rejects the file with a size-related error message.

TC-004 – Image Preprocessing

Type: Unit / Integration

Description: Verify preprocessing pipeline execution.

Steps:

1. Upload a valid image

Expected Result: The image is resized, normalized, and processed without errors.

TC-005 – Disease Classification

Type: Functional

Description: Verify classification output.

Steps:

1. Upload a diseased leaf image

Expected Result: The system returns a disease label and confidence score.

TC-006 – Health Score Validation

Type: Functional

Description: Verify health score range.

Steps:

1. Upload a valid image

Expected Result: Health score is between 0–100.

TC-007 – Nutrient Deficiency Indication

Type: Functional

Description: Verify deficiency suggestions.

Steps:

1. Upload an image with visible deficiency signs

Expected Result: System shows a suggested nutrient deficiency.

TC-008 – Result Page Display

Type: System / UI

Description: Verify result visualization.

Steps:

1. Complete analysis

Expected Result: All outputs (label, confidence, health score) are displayed correctly.

TC-009 – PDF Export

Type: Integration

Description: Verify PDF report generation.

Steps:

1. Click “Download PDF” after analysis

Expected Result: PDF is generated and downloaded successfully.

TC-010 – Invalid Image Handling

Type: Negative

Description: Verify behavior for corrupted input.

Steps:

1. Upload corrupted image

Expected Result: System displays error message.

TC-011 – Low Confidence Warning

Type: Functional

Description: Verify warning behavior.

Steps:

1. Upload unclear image

Expected Result: System displays low-confidence warning.

TC-012 – End-to-End Workflow

Type: System

Description: Verify full system flow.

Steps:

1. Upload → analyze → view results → download PDF

Expected Result: Entire process completes successfully.

TC-013 – Performance Test

Type: Performance

Description: Verify response time.

Steps:

1. Upload standard image

Expected Result: Result is generated within acceptable time ($\sim \leq 5$ seconds).

TC-014 – Repeated Input Consistency

Type: Reliability

Description: Verify consistency of results.

Steps:

1. Upload same image multiple times

Expected Result: Similar results are produced each time.

TC-015 – Data Privacy Check

Type: Security

Description: Verify temporary data handling.

Steps:

1. Upload image and complete analysis

Expected Result: Image is not permanently stored.